

BACnet Stack



An open source BACnet protocol stack for embedded systems

About this Project

[This BACnet protocol stack library](#) provides a BACnet application layer, network layer and media access (MAC) layer communications services. It is an open source, royalty-free library for an embedded system, Windows, Linux, or other operating system. [Example BACnet client and server applications](#) are included.

BACnet - A Data Communication Protocol for Building Automation and Control Networks - see bacnet.org. BACnet is a standard data communication protocol for Building Automation and Control Networks. BACnet is an open protocol, which means anyone can contribute to the standard, and anyone may use it. The only caveat is that the BACnet standard document itself is copyrighted by ASHRAE, and they sell the document to help defray costs of developing and maintaining the standard (just like IEEE or ANSI or ISO).

For software developers, the BACnet protocol is a standard way to send and receive messages on the wire containing data that is understood by other BACnet compliant devices. The BACnet standard defines a standard way to communicate over a number of wires, known as Data Link/Physical Layers: Ethernet, EIA-485, EIA-232, ARCNET, and LonTalk. The BACnet standard also defines a standard way to communicate using UDP, IP and HTTP (Web Services).

There are other open source projects for BACnet:

- [VTS](#) - visual test shell for Win32, used for Visually testing a BACnet implementation. It also includes a detailed network sniffer for BACnet messages, and the ability to send any BACnet services. The source code is in the public domain.
- [Wireshark](#) - an open source, cross platform protocol analyzer with BACnet support. The detailed BACnet support began in version 0.10.11 released on May 4, 2005 when Wireshark was known as Ethereal.
- [BACnet4Linux](#) - an LGPL BACnet application that requires Linux as the OS.
- [BACnet Firewall Router](#) -an application that combines BACnet routing capability with traffic management functions to carefully control access to building automation and control networks.
- [BACpypes](#) - a BACnet stack written in Python.
- [BACsharp](#) - a BACnet/IP stack written in C#.
- [BACnet4J](#) - a BACnet/IP stack written in Java that serves as the BACnet layer for [Mango](#). Mango is open source Machine-to-Machine software (aka Industrial Control, SCADA, HMI, or domotics).

There are also commercial BACnet protocol source code libraries for BACnet that are designed for embedded use:

- [Cimetrics](#)™ - has a source library called BACstac/32 as part of their BACNet Protocol Stack SDK.

- [Polarsoft](#) - has a protocol stack source library for embedded use called FreeRange™ and PolarSoft® FreeRange VSB (Very Small BACnet stack).
- [SCADA Engine](#) - The BACnet Linux Server is a complete BACnet Device running on the linux platform. The entire source code is available for custom applications and has been written in ANSI C which has been successfully ported to Unix, VxWorks etc.
- [BACnet Stack](#) - Chipkin automation systems BACnet stack is an application layer BACnet library for an embedded system and application development.

Licensing

Our BACnet protocol stack implementation is specifically designed for the embedded BACnet appliance, using a GPL with exception license (like [eCos](#)), which means that any changes to the core code that are distributed are made available, but the BACnet library can be linked to proprietary code without it becoming licensed under the GPL. See the [eCos license overview](#) for easy to read details about this exception to the GPL. The license does *not* require users to release the source code of any *applications* that are developed with this BACnet stack - only portions of the BACnet stack that have been modified. Note that those files in this BACnet stack that are expected to be modified are licensed using the [MIT License](#).

The text of the GPL exception included in each source file is as follows:

"As a special exception, if other files instantiate templates or use macros or inline functions from this file, or you compile this file and link it with other works to produce a work based on this file, this file does not by itself cause the resulting work to be covered by the GNU General Public License. However the source code for this file must still be made available in accordance with section (3) of the GNU General Public License."

The source code

The source code is written in C for portability, and includes unit tests (PC based unit tests) and example application code. Since the code is designed to be portable, it compiles with GCC as well as other compilers, such as Borland C++, Visual C++, MinGW, Code Warrior, IAR, or MicroChip C18. The source code is also designed to be high quality, readable, understandable, and easy to use.

The BACnet protocol is an ASHRAE/ANSI/ISO standard, so this library adheres to that standard. BACnet has no royalties or licensing restrictions, and [registration for a BACnet vendor ID](#) is free.

What the code does

The BACnet stack comes with example applications that can be run under Linux, Win32, RTOS-32, and just about any embedded microcontroller.

The BACnet stack includes unit tests can be run in a command shell on Linux using the unittest.sh script, or using individual .mak files. They were tested under [Debian GNU/Linux](#) and [Ubuntu Linux](#).

The BACnet stack was functionally tested using [VTS \(Visual Test Shell\)](#), another project hosted on SourceForge, as well as various BACnet controllers, BACnet workstations, and through BACnet routers. Some versions of the BACnet stack were BTL tested.

Demo BACnet Applications

Using a master Makefile in the project root directory, a dozen demo applications can be created that run under Linux or Win32. Linux supports BACnet Ethernet, BACnet/IP, or ARCNET data link layer for communication, and BACnet/IP is supported under Win32. BACnet Ethernet can also be used under Win32 with the [WinPcap library](#). Root privileges are required to run the Ethernet or ARCNET interfaces on Linux, but are not needed to run BACnet/IP. MS/TP support under Windows or Linux using a USB to RS-485 device is a work in progress.

To build all the demo applications under linux using the BACnet/IP datalink layer, use the familiar make command:

```
$ make clean all
```

You can also modify the Makefile variables from the command line to build with the BACnet MS/TP datalink layer, for example:

```
$ make BACDL_DEFINE=-DBACDL_MSTP=1 clean all
```

To build all the demo applications under Windows with the BACnet/IP datalink layer and utilizing the [MinGW tools](#), use the build.bat file provided in the root directory. Note that you may have to copy the mingw-make.exe to make.exe to enable the compile.

```
$ build.bat
```

To build all the demo applications under Windows with the BACnet/IP datalink layer and utilizing the [Borland tools](#), use the borland.bat file provided in the root directory.

```
$ borland.bat
```

To build all the demo applications under Windows using the BACnet/IP datalink layer utilizing the [Code::Blocks](#) tools, use the BACnetDemo.workspace file provided in the [demo](#) directory.

The demo application accept command line arguments. To specify an array index of ALL, use "-1". To make a priority optional, use "0". The applications also use environment variables to set datalink layer preferences. Use --help on the command line to see more options.

```
$ demo/server/bacsrv 123
BACnet Server Demo - Device #123
```

```
$ demo/readprop/bacrp
bacrp device-instance object-type object-instance property [index]
```

```
$ demo/writeprop/bacwp
bacwp device-instance object-type object-instance property priority index tag value [tag value...]
```

```
$ demo/readfile/bacarf
bacarf device-instance file-instance local-name
```

```
$ demo/writefile/bacawf
bacawf device-instance file-instance local-name
```

```
$ demo/reinit/bacrd
Usage: bacrd device-instance state [password]
```

Send BACnet ReinitializeDevice service to device.

```
$ demo/whohas/bacwh
```

Usage: bacwh object-type object-instance | object-name

Send BACnet WhoHas request to devices, and wait for responses.

```
$ demo/dcc/bacdcc
```

Usage: bacdcc device-instance state timeout [password]

Send BACnet DeviceCommunicationControl service to device.

```
$ demo/timesync/bacts
```

Received TimeSynchronization Request

2006/8/30 07:10:45.00

```
$ demo/ucov/bacucov
```

Usage: bacucov pid device-id object-type object-instance time property tag value [priority]
[index]

```
$ demo/whois/bacwi
```

Usage: bacwi device-instance | device-instance-min device-instance-max

Send BACnet WhoIs request to devices, and wait for responses.

The device-instance can be 0 to 4194303, or -1 for ALL.

The device-instance can also be specified as a range.

The demos can be compiled under Win32 using [MinGW - Minimalist GNU for Windows](#), [Borland C++](#), or [Microsoft Visual C++](#), which are free command line compilers. Be sure to pick up the free [patches \(service packs\) for the Borland C++ compiler](#) ([SP1](#), [SP2](#)), as well as the free turbo debugger. It is also possible to create Win32 projects using the free [Visual Studio Express Edition](#) after downloading the platform development kit for your operating system. You can also use [MinGW - Minimalist GNU for Windows](#) which comes with Code::Blocks. I frequently use [Code::Blocks](#) for compiling the unit tests using the MinGW compiler and created some Code::Block projects for some of the demos. I have also used [Code::Blocks](#) with the Borland C++ compiler and it successfully compiles and runs the code.

[BACnet-Tools](#) - the example applications are compiled for Windows and can be downloaded from SourceForge.

To build the demo applications under Linux, such as [Ubuntu](#), you may need to install some build tools.

```
$ sudo apt-get install build-essential subversion-tools
```

Example BACnet Server Ported to Various Architectures

There is a Makefile in the [ports/rtos32](#) directory, and a sample application that runs under [RTOS-32](#). It currently uses the BACnet/IP data link layer for communication, and also has an MS/TP datalink layer sample application. It compiles using [Borland C++](#).

There is a project in the [ports/pic18f6720](#) directory, and a sample application that can be built using [MP-Lab](#) and the Microchip compiler MCC18. The datalink layer uses BACnet MS/TP and the example uses several different objects and services.

There is a project in the [ports/at91sam7s](#) directory for the AT91SAM7S-EK demo board. There is a server application that can be built using the GNU ARM tools, such as [GNU Toolchain for ARM](#), [GNU ARM](#), [WinARM](#), [YAGARTO](#), or [Rowley Crossworks for ARM](#) toolchains on the Windows platform.

The datalink layer uses BACnet MS/TP and the example uses several different objects and services.

There is a project in the [ports/atmega168](#) directory, and a sample server application that can be built using [GCC-AVR](#) or [WinAVR](#) for Atmel AVR series of microcontrollers. There is also a project in the [ports/bdk-atxx4-mstp](#) directory which works on the [ATmega644 based BACnet Development Kit](#).

Both projects use the BACnet MS/TP datalink layer and the example uses several different objects and services. In addition to the free tools listed above, the AVR projects can be developed using the commercial [Rowley Crossworks for AVR](#) on the Windows, MAC OS X, Linux, or Solaris platform. CrossWorks is a complete development environment. The AVR projects can also be developed using the commercial [IAR Embedded Workbench for Atmel AVR](#) environment on Windows.

There is a project in the [ports/stm32f10x](#) directory, and a sample server application that can be built using [IAR Embedded Workbench for ARM](#). It was written for the 2011 [STM32 Design Challenge](#), and was one of the finalists. The Target is an ARM Cortex-M3 microcontroller, and the design utilizes the [CMSIS](#) and STM32 Peripheral Driver Library. The port uses the BACnet MS/TP datalink layer and the example uses several different objects and services.

BACnet services supported matrix

The BACnet stack currently implements the following services listed in the the table. We plan to add the rest of the services as we go. With the services that are implemented, you could build a BACnet device that meets the standardized profile for a BACnet Smart Sensor, BACnet Smart Actuator, or a BACnet Application Specific Controller.

BACnet Service	Initiate	Execute
Who Is	Yes	Yes
I Am	Yes	Yes
Who Has	Yes	Yes
I Have	Yes	Yes
Read Property	Yes	Yes
Write Property	Yes	Yes
Device Communication Control	Yes	Yes
ReinitializeDevice	Yes	Yes
Atomic Read File	Yes	Yes
Atomic Write File	Yes	Yes
Time Synchronization	Yes	Yes
UTC Time Synchronization	Yes	Yes
Subscribe COV	Yes	Yes
Confirmed COV Notification	Yes	Yes
Unconfirmed COV Notification	Yes	Yes
Read Property Multiple	Yes	Yes
Read Property Conditional	-	-
Read Range	-	Yes
Write Property Multiple	-	Yes
Get Alarm Summary	-	Yes

Get Event Information	-	Yes
Get Enrollment Summary	-	-
Acknowledge Alarm	-	Yes
Confirmed Event Notification	-	Yes
Unconfirmed Event Notification	-	Yes
Unconfirmed Text Message	-	-
Confirmed Text Message	-	-
Add List Element	-	-
Remove List Element	-	-
Create Object	-	-
Delete Object	-	-
Unconfirmed Private Transfer	Yes	Yes
Confirmed Private Transfer	Yes	Yes
VT Open	-	-
VT Data	-	-
VT Close	-	-

BACnet Objects

The BACnet stack currently implements an example Device Object, and handles all of the ReadProperty and ReadPropertyMultiple inquiries for the required Device Object properties. The stack handles Who-Is inquiries with an I-Am, WhoHas with I-Have, and handles reject messages for services not currently supported or implemented by your device. There is built-in handling for DeviceCommunicationControl.

The example handlers interact with example objects by way of the Device object. There are example objects for the developer to use as a template when customizing the objects for their device.

File Objects are conditionally included in the demonstration applications. The files can be accessed using WriteProperty, ReadProperty, Who-Has, AtomicWriteFile, or AtomicReadFile services.

BACnet Object	Code Example
Accumulator	-
Analog Input	Yes
Analog Output	Yes
Analog Value	Yes
Averaging	-
Binary Input	Yes
Binary Output	Yes
Binary Value	Yes
Calendar	-
Command	-
Device	Yes
Event Enrollment	-

File	Yes
Group	-
Life Safety Point	Yes
Life Safety Zone	-
Loop	-
Multi-state Input	Yes
Multi-state Output	Yes
Multi-state Value	-
Notification Class	Yes
Program	-
Pulse Converter	-
Schedule	-
Trend Log	Yes
Access Door	-
Event Log	-
Load Control	Yes
Structured View	-
Trend Log Multiple	-
Access Point	-
Access Zone	-
Access User	-
Access Rights	-
Access Credential	-
Credential Data Input	-
CharacterString Value	-
DateTime Value	-
Large Analog Value	-
BitString Value	-
OctetString Value	-
Time Value	-
Integer Value	-
Positive Integer Value	-
Date Value	-
DateTime Pattern Value	-
Time Pattern Value	-
Date Pattern Value	-
Network Security	-
Global Group	-
Notification Forwarder	-
Alert Enrollment	-
Channel	-

Getting Involved

If you want to help out on this project, join the [developers mailing list](#), introduce yourself, and tell us what you would like to do. If you are trying to implement a BACnet device or service using this project, you are welcome to join the same [developers mailing list](#) as well.

More details about the project can be found on the [BACnet Source Forge Project Page](#)

There is documentation that [describes the mechanisms in the BACnet Stack](#). I wrote up some [answers to some frequently asked questions](#). Of course, there are a handful of text files in the doc directory of the project with more useful information.

[BACnet Stack released files download](#)

You can get the latest BACnet protocol stack source code using the [Subversion](#) version control system. The main development branch is at: <http://svn.code.sf.net/p/bacnet/code/trunk/bacnet-stack/>. This has the absolute latest code and features. Anyone doing development on the BACnet protocol stack should be using this branch. The stable releases are at: <http://svn.code.sf.net/p/bacnet/code/tags/>. This matches the released version downloadable through SourceForge. Anyone doing project development using the BACnet protocol stack should be using the tags branch and an appropriate version tag. The custom and vendor branches are at: <http://svn.code.sf.net/p/bacnet/code/branches/>. The [JBennet branch](#) includes a derivative project that is designed for a BACnet Client and includes segmentation.

To get the source code from the trunk repository, use "svn checkout", e.g.

```
svn checkout https://svn.code.sf.net/p/bacnet/code/trunk/bacnet-stack/  
or for anonymous checkout:  
svn checkout http://svn.code.sf.net/p/bacnet/code/trunk/bacnet-stack/  
or for the stable releases:  
svn checkout http://svn.code.sf.net/p/bacnet/code/tags/bacnet-stack-0-7-1/  
or for the JBennet branch:  
svn checkout http://svn.code.sf.net/p/bacnet/code/branches/jbennet/
```

BACnet Developer Resources

There are a number of resources that can help you develop a BACnet product or project.

- [VTS](#) - visual test shell for Win32, used for Visually testing a BACnet implementation. It also includes a detailed network sniffer for BACnet messages, and the ability to send any BACnet services. The source code is in the public domain.
- [Wireshark](#) - an open source, cross platform protocol analyzer with BACnet support. The detailed BACnet support in began in version 0.10.11 released on May 4, 2005 when Wireshark was known as Ethereal. Also, BACnet MS/TP capture support was added in Wireshark version 1.1.0. The [demo application mstpcap](#) can be used to capture BACnet MS/TP using an RS-485 converter, and save the data to a capture file that can be viewed with Wireshark.
- [Ubuntu Linux](#) or [Debian Linux](#) - my development platforms of choice. Linux makes a great development platform because all the necessary development tools are included.
- [Code::Blocks](#) - a free cross-platform open source C/C++ IDE. Includes the MinGW compiler for

Win32.

- Win32 development can use [Borland C++](#) or [Microsoft Visual C++](#), both of which are free (as in beer) command line compilers. Be sure to pick up the free [patches \(service packs\) for the Borland C++ compiler](#) ([SP1](#), [SP2](#)), as well as the free turbo debugger.

[BACnet Developer Help](#)

[BACnet International Developer Resources](#)

Products and Projects that use this BACnet Stack

Did you develop a product using this BACnet stack? [Let us know](#), and you can get a little recognition for your hard work!

[BACnet Development Kit](#) - An Atmel ATmega644 based development kit designed to kickstart BACnet MS/TP development.

[Digital Lighting Management](#) - offers connectivity without complexity for remote system management and control of lights and plug loads. Just one device in each room network has to be connected to a BACnet MS/TP segment for centralized control.

[CSWorks](#) - a development framework for building web-based HMI (Human Machine Interface), SCADA (Supervisory Control And Data Acquisition), EMI (Enterprise Manufacturing Intelligence) and M2M (Machine to Machine) software applications.

[SCInterface™ = Sensor Control Interface](#) - middleware platform for managing legacy and modern-day sensors through a centralized interface.

[BACnetSim](#) - a portable implementation of the BACnet data communication protocol. BACnetSim is meant for embedded devices and use MS/TP as the media access layer. BACnetSim is a fork of bacnet-stack-0.0.1

[Building Controls Virtual Test Bed](#) - a software environment that allows coupling different simulation programs including Dymola, EnergyPlus, MATLAB/Simulink and Radiance for co-simulation, and to couple these programs to control systems.

[The coelostat in the Linde + Robinson Laboratory](#) at Caltech.



ASHRAE® and BACnet® are registered trademarks of the American Society of Heating, Refrigerating and Air-Conditioning Engineers, Inc., 1791 Tullie Circle NE, Atlanta, GA 30329.

Website updated 15-October-2013 by [Steve Karg](#).